React Testing Library: Practical Guide

4 min read Updated 24 Mar 2026 Topic 41 of 62

9.2 - React Testing Library: Practical Guide

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"Write a test for a counter component"	🟡🔴	★★★★☆
"How do you query elements in RTL?"	🟡	★★★★☆
"What is the difference between getBy, queryBy, and findBy?"	🟡🔴	★★★★☆
"How do you test async behavior?"	🔴	★★★☆☆

Setting Up a Test

Every test follows the same three phases: **Arrange** (render the component), **Act** (simulate user interactions), **Assert** (check the expected outcome).

JSX Copy

```jsx
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

describe('Counter', () => {
  test('renders initial count of 0', () => {
    render(<Counter />);
    expect(screen.getByText('Count: 0')).toBeInTheDocument();
  });

  test('increments count when + button is clicked', () => {
    render(<Counter />);
    fireEvent.click(screen.getByRole('button', { name: /increment/i }));
    expect(screen.getByText('Count: 1')).toBeInTheDocument();
  });

  test('disables decrement button when count is 0', () => {
    render(<Counter />);
    expect(screen.getByRole('button', { name: /decrement/i })).toBeDisable
  });
});
```

Query Methods

RTL provides three families of queries, each for a different scenario:

Query	Returns	Throws if not found?	Use whe
`getBy ...`	The element	Yes	Element
`queryBy ...`	The element or `null`	No	Checkin

jsx

```jsx
// Element SHOULD exist
const heading = screen.getByRole('heading', { name: /welcome/i });

// Element SHOULD NOT exist
expect(screen.queryByText('Error')).not.toBeInTheDocument();

// Element will APPEAR after async operation
const userData = await screen.findByText('Alice');
```

Query Priority (what to reach for first)

1. `getByRole` (most accessible, mirrors how screen readers see the page)
2. `getByLabelText` (for form inputs)
3. `getByPlaceholderText` (for inputs without labels)
4. `getByText` (for non-interactive text content)
5. `getByTestId` (last resort, when nothing else works)

Testing User Interactions

`fireEvent` works for basic cases, but `userEvent` from `@testing-library/user-event` is more realistic. It simulates full interaction sequences (focus, keystrokes, blur) rather than just dispatching a single event.

jsx

```jsx
```

```jsx
test('typing in search input filters the list', async () => {
  const user = userEvent.setup();
  render(<SearchableList items={['Apple', 'Banana', 'Cherry']} />);

  const input = screen.getByPlaceholderText('Search...');
  await user.type(input, 'ban');

  expect(screen.getByText('Banana')).toBeInTheDocument();
  expect(screen.queryByText('Apple')).not.toBeInTheDocument();
  expect(screen.queryByText('Cherry')).not.toBeInTheDocument();
});
```

Testing Async Components

Components that fetch data need `findBy` queries (which wait for elements to appear) and `waitFor` for more complex assertions.

jsx

JSX Copy

```jsx
test('loads and displays users', async () => {
  render(<UserList />);

  expect(screen.getByText('Loading...')).toBeInTheDocument();

  const userItem = await screen.findByText('Alice');
  expect(userItem).toBeInTheDocument();

  expect(screen.queryByText('Loading...')).not.toBeInTheDocument();
});
```

Testing Forms

JSX Copy

```jsx
test('submits form with entered values', async () => {
  const handleSubmit = jest.fn();
  const user = userEvent.setup();

  render(<LoginForm onSubmit={handleSubmit} />);

  await user.type(screen.getByLabelText('Email'), 'alice@test.com');
  await user.type(screen.getByLabelText('Password'), 'secret123');
  await user.click(screen.getByRole('button', { name: /login/i }));

  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'alice@test.com',
    password: 'secret123',
  });
});

test('shows validation error for empty email', async () => {
  const user = userEvent.setup();
  render(<LoginForm onSubmit={jest.fn()} />);

  await user.click(screen.getByRole('button', { name: /login/i }));

  expect(screen.getByText('Email is required')).toBeInTheDocument();
});
```

Snapshot Testing

Snapshot tests capture the rendered output and compare it against a saved file. If the output changes unexpectedly, the test fails.

jsx

JSX Copy

```
  expect(container).toMatchSnapshot();
});
```

Caution: Snapshot tests break easily with any UI change, leading to developers blindly updating snapshots. Use them sparingly for stable, static components, not dynamic ones.

Common Mistakes

Mistake 1: Testing implementation details. Don't test internal state or whether specific hooks were called. Test what the user sees.

Mistake 2: Using `getBy` to check something doesn't exist. `getBy` throws an error if the element isn't found. Use `queryBy` instead.

Mistake 3: Not using `findBy` for async content. If your component fetches data, use `findBy` or `waitFor`. `getBy` fails because the content hasn't loaded yet.

Mistake 4: Over-relying on `getByTestId`. It couples tests to implementation. Prefer semantic queries like `getByRole` and `getByLabelText`.

QUICK CHECK — no marks, just for you.

1. You want to assert that an error message is NOT rendered on the screen. Which RTL query family should you use, and why?

- ○ findBy..., because it resolves to null once the timeout expires
- ○ getBy..., because it returns null when the element is absent
- ○ queryBy..., because it returns null instead of throwing when the element is absent
- ○ getAllBy..., because it returns an empty array when nothing matches

2. According to the lesson, what makes userEvent from @testing-library/user-event more realistic than fireEvent?

- ◯ It runs the interaction in a real browser rather than in jsdom
- ◯ It automatically wraps every assertion in waitFor so async updates are handled
- ◯ It simulates full interaction sequences such as focus, keystrokes and blur, instead of dispatching a single event
- ◯ It bypasses React's synthetic event system and dispatches native DOM events directly

Check answer

3. A component shows 'Loading...' and then renders 'Alice' after fetching data. A test using screen.getByText('Alice') immediately after render() fails. What is the correct fix per the lesson?

- ◯ Wrap the getByText call in queryByText, since queryBy retries until the element exists
- ◯ Add a testid to the element and use getByTestId, which is not affected by timing
- ◯ Await screen.findByText('Alice'), since findBy waits for async content to appear
- ◯ Use getAllByText('Alice') so the query waits for at least one match to render

Check answer

React.js Interview Kit

LEARN MORE

ARTICLES & DOCS

- **React Testing Library — intro**
  testing-library.com
- **Which query should I use?**
  testing-library.com

VIDEOS



React Testing Library — introduction
YouTube



RTL & Jest crash course
YouTube



Start testing in ReactJS
YouTube

Hand-picked references for this lesson.